# ■ Functional Programming

Python — Functions, Lambda, Map, Filter, Reduce & Decorators

| # | Topic |
|---|---|
| 1 | Pure Functions |
| 2 | Higher-Order Functions |
| 3 | Defining Functions  (def) |
| 4 | Code Reusability |
| 5 | Prime Number Function — Example |
| 6 | *args  &  **kwargs |
| 7 | Lambda Functions |
| 8 | map() |
| 9 | filter() |
| 10 | reduce() |
| 11 | Decorators |

# 1 · Pure Functions

- Always produce the **same output** for the same input
- Have **no side effects** — don't modify external state
- Easier to test, debug, and reason about

| Characteristic | Meaning |
|---|---|
| Deterministic | Same input always → same output |
| No side effects | Doesn't modify variables outside itself |
| Referentially transparent | Can replace call with its return value |

# 2 · Higher-Order Functions

A **Higher-Order Function** is one that does at least one of:

- **Returns** another function as its result
- **Takes** a function as a parameter / argument

■ *map(), filter(), reduce(), and decorators are all higher-order functions.*

# 3 · Defining Functions (def)

Python

```python
def function_name(parameters):
    # function body
    return value
```

| Part | Description |
|---|---|
| def | Keyword that starts a function definition |
| function_name | Name you give to the function (snake_case by convention) |
| parameters | Input values the function accepts (optional) |
| return | Sends a value back to the caller (optional — returns None if omitted) |

# 4 · Code Reusability

- Write once, use many times — avoids repeating the same logic
- Makes code shorter, cleaner, and easier to maintain
- Bugs only need to be fixed in one place
- Functions can be imported and shared across files/modules

# 5 · Example — Prime Number Function

A number is **prime** if it is greater than 1 and divisible only by 1 and itself.

Python

```python
def prime(num):
    if num <= 1:
        return False
    for i in range(2, num):
        if num % i == 0:
            return False
    return True

prime(7)    # True
prime(10)   # False
```

■ *Optimisation: use range(2, int(num**0.5)+1) instead of range(2, num) — you only need to check up to the square root.*

# 6 · *args & **kwargs

| Parameter | Meaning | Receives |
|---|---|---|
| *args | Variable number of positional arguments | Tuple of values |
| **kwargs | Variable number of keyword arguments | Dict of key=value pairs |

## *args example

Python

```python
def addition(*args):
    result = sum(args)
    return result

addition(2, 3, 4)   # 9  — any number of arguments
```

## **kwargs example

Python

```python
def display_info(**kwargs):
    for key, value in kwargs.items():
        print(f'{key}: {value}')

display_info(name='Alice', age=30)
# name: Alice
# age: 30
```

■ *Tip: You can combine both: def func(*args, **kwargs) — args must come before kwargs.*

# 7 · Lambda Functions

A **lambda** is an anonymous (nameless), single-expression function.

Syntax

```
lambda argument : expression
```

Python

```python
# Regular function
def add(a, b):
    return a + b

# Equivalent lambda
add = lambda a, b: a + b

add(7, 9)   # 16
```

| Feature | def | lambda |
|---|---|---|

| | | |
|---|---|---|
| Name | Has a name | Anonymous (no name) |
| Body | Multiple statements allowed | Single expression only |
| return | Explicit return | Implicit (expression value is returned) |
| Use case | General-purpose functions | Short throwaway / inline functions |

## 8 · map()

**map()** applies a function to **every element** of an iterable and returns a map object.

Syntax

```
map(function, iterable)
```

Python

```python
list1 = [1, 2, 3, 4]

def square(n):
    return n ** 2

result = list(map(square, list1))
# [1, 4, 9, 16]

# Same thing with lambda (more concise)
result = list(map(lambda n: n ** 2, list1))
# [1, 4, 9, 16]
```

■ *map() returns a lazy map object — wrap it in list() to see the results.*

## 9 · filter()

**filter()** keeps only elements for which the function returns **True**.

Syntax

```
filter(function, iterable)
```

Python

```python
nums = [1, 2, 3, 4, 5, 6, 7]

evens = filter(lambda x: x % 2 == 0, nums)
list(evens)   # [2, 4, 6]
```

| Function | Acts on | Returns |
|---|---|---|
| map() | Every element | Transformed value for each element |
| filter() | Every element | Only elements where function returns True |

# 10 · reduce()

**reduce()** cumulatively applies a function to reduce an iterable to a **single value**. It must be imported from `functools`.

```python
from functools import reduce

nums = [1, 2, 3, 4, 5, 6, 7]
result = reduce(lambda x, y: x + y, nums)
```

## How reduce() works step-by-step

| Step | Calculation | Running Total |
|---|---|---|
| 1 | 1 + 2 | 3 |
| 2 | 3 + 4 | 7 |
| 3 | 7 + 5 | 12 |
| 4 | 12 + 6 | 18 |
| 5 | 18 + 7 | 25 ← final result |

■ *reduce() processes pairs left-to-right, accumulating a single result. The list [1,2,3,4,5,6,7] sums to 25.*

# 11 · Decorators

A **decorator** is a higher-order function that **wraps another function** to extend its behaviour — without changing its source code.

| Concept | Description |
|---|---|
| @decorator_name | Syntax sugar — equivalent to: func = decorator(func) |
| wrapper() | Inner function that runs before/after the original function |
| return wrapper | The decorator returns the wrapper to replace the original |

## Decorator — Example

Python

```python
def decorator(func):
    def wrapper():
        print('start')
        func()
        print('end')
    return wrapper

@decorator
def say_hello():
    print('Hello!')

say_hello()
# Output:
# start
# Hello!
# end
```

The `@decorator` syntax above is equivalent to writing:

Python

```python
def say_hello():
    print('Hello!')

say_hello = decorator(say_hello)    # manual decoration
```

■ *Decorators are used heavily in frameworks like Flask (@app.route), Django, and for logging, timing, authentication, and caching.*

# Quick Reference — Functional Tools

| Tool | Purpose | Import needed? |
|---|---|---|
| def | Define a named function | No |
| return | Send a value back to caller | No |

| | | |
|---|---|---|
| lambda | Inline anonymous function | No |
| *args | Accept any number of positional args | No |
| **kwargs | Accept any number of keyword args | No |
| map() | Apply function to every element | No |
| filter() | Keep elements where function returns True | No |
| reduce() | Collapse iterable to single value | from functools import reduce |
| @decorator | Wrap a function to extend its behaviour | No |

Python

```python
# All three together — functional style pipeline
from functools import reduce

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

result = reduce(
    lambda x, y: x + y,
    map(lambda n: n ** 2,
        filter(lambda n: n % 2 == 0, nums))
)  # sum of squares of even numbers: 4+16+36+64+100 = 220
```

■ *Tip: Chaining map → filter → reduce is a classic functional programming pattern — readable and avoids intermediate variables.*